



Hacettepe University
Computer Engineering

MIPS ASSEMBLER PROJECT

WIN!

PRIZES!

SUBMISSION DEADLINE:
SATURDAY 16/05/2026 AT 23:59

BBM234 Computer Organization
Spring 2026

Project #3

The MIPS Assembler Project

BBM234 Computer Organization
Department of Computer Engineering
Hacettepe University

Spring 2026

Programming language:	<i>C</i>
Submission:	Saturday, 16/05/2026 at 23:59 via https://submit.cs.hacettepe.edu.tr/
Inputs:	Three file paths, given as command-line arguments
Output:	Written to standard output (stdout)

1 Background

An *instruction set*, together with its *instruction set architecture (ISA)*, forms the interface between a computer's software and its hardware. The ISA specifies the parts of the computer architecture that are visible to the programmer: native data types, instructions, registers, addressing modes, memory architecture, interrupt and exception handling, and external I/O. An ISA includes a specification of the set of opcodes (machine instructions) and the native commands implemented by a particular processor.

An *opcode* (abbreviated from *operation code*) is the portion of a machine-language instruction that specifies the operation to be performed. Each instruction performs a very specific task: a load, a store, a jump, a branch, or an ALU operation on a unit of data held in a CPU register or in memory. Every program directly executed by a CPU is, ultimately, a sequence of such instructions.

1.1 RISC and the MIPS ISA

Reduced Instruction Set Computing (RISC) is a CPU design philosophy based on the insight that a simplified instruction set can deliver higher performance when combined with a microprocessor architecture that executes each instruction in fewer cycles.

MIPS (Microprocessor without Interlocked Pipeline Stages) is a RISC ISA that was originally designed for general-purpose computing and is today widely used in embedded systems such as residential gateways, routers, and game consoles. A MIPS processor reads only binary; however, writing programs directly in binary is impractical for humans. To bridge this gap, programmers use *assembly language*, which uses mnemonics, register names, and operands instead of raw binary digits. Assembly source code is then translated to executable machine code by an *assembler*.

A typical MIPS instruction definition has the following form:

ADD — Add (with overflow)

Description: Adds two registers and stores the result in a register.

Operation: $\$d = \$s + \$t$; `advance_pc(4)`;

Syntax: `add $d, $s, $t`

Encoding: 0000 00ss ssst tttt dddd d000 0010 0000

Each lowercase letter in the encoding (*s*, *t*, *d*, *i*) is a placeholder for a bit field that must be filled in based on the operands provided in the assembly source.

2 Problem Definition

Your task is to implement an **assembler** that translates a given MIPS-assembly program into the corresponding machine code.

You are expected to complete a total of **six functions** provided as TODOs in the starter code: `int_to_binary()` and `binary_to_hex()` in `binary.c`, and `encode_r_type()`, `encode_i_type()`, `encode_branch()`, and `encode_j_type()` in `assembler.c`. Detailed instructions for each function are included in the starter code. Your program must be able to:

1. Read and decode the register table given in `registers.txt`.
2. Read and parse the instruction definitions (syntax and encoding) given in `instructions.txt`.
3. Assemble each line of the program in `program.txt` and print the corresponding machine code to standard output, in both *binary* and *hexadecimal* representations.

The three input files are passed as command-line arguments, in the following order:

```
<your_executable> registers.txt instructions.txt program.txt
```

3 Program Layout and Branch / Jump Semantics

For this assignment, the program described in `program.txt` is treated as if it were loaded into memory starting at byte address `0x00000000`. Each MIPS instruction is exactly 4 bytes long. Therefore:

- The first line of `program.txt` sits at $PC = 0$.
- The second line sits at $PC = 4$.
- In general, the k -th line (counting from $k = 0$) sits at $PC = 4 \cdot k$.

For most instructions (R-type, and non-branch I-type loads, stores, and immediate ALU operations), the operands are encoded *directly* as their numeric values. Two instruction families, however, require an extra translation step that depends on the program counter PC.

3.1 Branch Instructions (e.g. `beq`)

Branch instructions do not store the target address directly. Instead, they store a **relative offset**.

In this assignment:

- Each instruction is 4 bytes.
- The k -th line is at $PC = 4k$.

For an instruction:

`beq $s, $t, target`

the encoded value is:

$$\text{offset} = \frac{\text{target} - (\text{PC} + 4)}{4}$$

Interpretation. The branch stores *how many instructions to move* relative to the next instruction.

Example (forward branch):

At PC = 8:

`beq $t0, $zero, 16`

$$\frac{16 - (8 + 4)}{4} = 1$$

So the encoded offset is 1.

Example (backward branch):

At PC = 12:

`beq $t0, $zero, 0`

$$\frac{0 - (12 + 4)}{4} = -4$$

Encoded as 0xffffc (two's complement).

3.2 Jump Instructions (j)

Jump instructions encode the target directly (unlike branches).

For:

`j target`

the encoded value is:

$$\frac{\text{target}}{4}$$

Interpretation. The jump stores the *instruction index*, not the byte address.

Example:

`j 8`

$$8/4 = 2$$

So the encoded value is 2.

Assumption (Simplified Model). In a real assembler, branch and jump instructions typically use **labels** (e.g. `beq $s, $t, loop`), which must be resolved to addresses by performing multiple passes over the program. In this assignment, we **do not require label resolution**. Instead, all branch and jump targets are given directly as **numeric byte addresses**:

- Branch and jump instructions use **numeric targets only**.
- You will **not** encounter labels (e.g. `beq $s, $t, loop`).
- You are **not expected** to implement label resolution or symbol tables.

Your task is therefore limited to **encoding** these values correctly, not computing them from symbolic labels.

4 Input File Specifications

4.1 Register File — `registers.txt`

In MIPS, several registers are available for storing data. General-purpose registers are denoted with a dollar sign (\$). The registers relevant to this assignment are:

Number	Name	Purpose
2 – 3	\$v0 – \$v1	Return values from functions
4 – 7	\$a0 – \$a3	Arguments to functions (not preserved by subprograms)
8 – 15	\$t0 – \$t7	Temporary data (not preserved by subprograms)
16 – 23	\$s0 – \$s7	Saved registers (preserved by subprograms)

This information is provided in `registers.txt`, where each line follows the format:

```
<register_number><TAB><register_name>
```

Your program will read this file and use it to translate register names into their numeric values when assembling instructions.

4.2 Instruction File — `instructions.txt`

The instruction definitions are provided in `instructions.txt`. Each line of this file follows the format:

```
<single-space-separated instruction syntax><TAB><encoding>
```

The *encoding* is a 32-bit binary template, written in groups of four bits separated by spaces. Lowercase letters within the encoding are placeholders for bits that depend on the operands of the instruction:

- **s** — bits of register `$s` (5 bits)
- **t** — bits of register `$t` (5 bits)
- **d** — bits of register `$d` (5 bits)
- **i** — bits of an immediate / offset value (**variable length**)

Important. The width of each placeholder field is determined *by the encoding itself*, not by your code. For example, an immediate field may be 16 bits long in one instruction and 26 bits long in another. Your assembler must respect the width given in the template.

A more comprehensive MIPS instruction reference is available at:
https://mathcs.holycross.edu/~csci226/MIPS/MIPS_InstructionReference.html

Key design principle. Your assembler must be **fully driven by `instructions.txt`**.

- You should **not hard-code** any instruction formats or field positions.
- Every instruction must be parsed and encoded **based only on the syntax and encoding provided in the file**.

In other words, your program should work correctly for *any* instruction definition that follows the given format, even if you have never seen it before, but *only for the instructions provided in the input file* — **not the full MIPS instruction set**.

4.3 Program File — `program.txt`

The `program.txt` file contains the MIPS-assembly program to be translated. Each line is exactly one instruction. Your assembler must process the lines in order and produce the machine code for each.

5 Output Specification

For each line of the input program, your program must print **exactly two lines** to standard output:

1. The 32-bit binary representation of the assembled instruction — exactly 32 characters consisting only of 0 and 1, with no spaces.
2. The hexadecimal representation, prefixed with `0x`, written in **lowercase** hex digits and zero-padded to exactly 8 hex digits (so the line is exactly 10 characters: `0x` followed by 8 hex digits).

The output must contain **no** extra whitespace, **no** leading or trailing blank lines, and **no** additional commentary!

6 Sample Input and Expected Output

6.1 Sample `registers.txt`

```
2 $v0
3 $v1
4 $a0
5 $a1
6 $a2
7 $a3
8 $t0
9 $t1
10 $t2
11 $t3
12 $t4
13 $t5
14 $t6
15 $t7
16 $s0
17 $s1
18 $s2
19 $s3
20 $s4
21 $s5
22 $s6
23 $s7
```

6.2 Sample instructions.txt

```
add $d, $s, $t 0000 00ss ssst tttt dddd d000 0010 0000
and $d, $s, $t 0000 00ss ssst tttt dddd d000 0010 0100
j imm 0000 10ii iiii iiii iiii iiii iiii iiii
jr $s 0000 00ss sss0 0000 0000 0000 0000 1000
lw $t, offset($s) 1000 11ss ssst tttt iiii iiii iiii iiii
mfhi $d 0000 0000 0000 0000 dddd d000 0001 0000
multu $s, $t 0000 00ss ssst tttt 0000 0000 0001 1001
slt $d, $s, $t 0000 00ss ssst tttt dddd d000 0010 1010
slti $t, $s, imm 0010 10ss ssst tttt iiii iiii iiii iiii
beq $s, $t, offset 0001 00ss ssst tttt iiii iiii iiii iiii
```

6.3 Sample program.txt

The sample program below is designed to exercise every instruction category covered by this assignment: R-type, non-branch I-type, branch, and J-type. Recall from Section 3 that the first line is at PC = 0, the second at PC = 4, and so on.

```
add $t1, $t2, $t3
slti $t0, $t1, -10
lw $s2, -8($t1)
slt $t4, $t1, $t2
and $t0, $t1, $t5
beq $t0, $s0, 4
multu $t1, $t2
slti $v0, $a0, 256
lw $a1, 100($s1)
beq $a0, $v0, 60
mfhi $t0
jr $s3
j 12
```

6.4 Expected Standard Output

```
00000001010010110100100000100000
0x014b4820
00101001001010001111111111110110
0x2928fff6
1000110100110010111111111111000
0x8d32fff8
00000001001010100110000000101010
0x012a602a
00000001001011010100000000100100
0x012d4024
0001000100010000111111111111011
0x1110fffb
000000010010101000000000000011001
0x012a0019
00101000100000100000000010000000
0x28820100
100011100010010100000000001100100
0x8e250064
00010000100000100000000000000101
0x10820005
0000000000000000100000000010000
0x00004010
00000010011000000000000000001000
0x02600008
0000100000000000000000000000011
0x08000003
```

7 Worked Examples

7.1 R-type: add \$t1, \$t2, \$t3

Looking up the syntax and encoding of `add` in `instructions.txt`:

- Syntax: `add $d, $s, $t`
- Encoding: `0000 00ss ssst tttt dddd d000 0010 0000`

Matching operands to placeholders, in the order they appear in the syntax:

- `$d = $t1` → 9 → 01001
- `$s = $t2` → 10 → 01010
- `$t = $t3` → 11 → 01011

Substituting each placeholder with the corresponding bit value:

000000 01010 01011 01001 00000 100000

yields the 32-bit machine code `00000001010010110100100000100000`, or `0x014b4820` in hexadecimal. (R-type encodings are independent of PC, so this instruction would produce the same machine code regardless of where it appeared in `program.txt`.)

7.2 Instructions with Immediate Fields

For `lw $t, offset($s)` with encoding `1000 11ss ssst tttt iiii iiii iiii iiii`, the trailing `i` field is a *16-bit* immediate. For `j imm` with encoding `0000 10ii iiii iiii iiii iiii iiii iiii`, the immediate field is *26 bits*. Your assembler must:

- Determine each field's bit-width by counting consecutive placeholder characters in the encoding (after stripping the spaces).
- Convert the integer value to that exact number of bits.

For most non-branch I-type instructions (e.g. `lw`, `slti`), the immediate value from the assembly source is encoded *directly* at the appropriate bit width. Negative immediates are represented in two's complement.

7.3 Branch: beq \$t0, \$s0, 0 at PC = 12

Suppose this instruction is the 4th line of `program.txt`, so `PC = 12`.

Looking up the syntax and encoding of `beq` in `instructions.txt`:

- Syntax: `beq $s, $t, offset`
- Encoding: `0001 00ss ssst tttt iiii iiii iiii iiii`

Matching operands to placeholders, register fields are encoded directly:

- `$s = $t0` → 8 → 01000
- `$t = $s0` → 16 → 10000

The third operand, 0, is the **target byte address**. Per Section 3, the value placed in the 16-bit `i` field is

$$\frac{0 - 12 - 4}{4} = \frac{-16}{4} = -4$$

The decimal value `-4` in 16-bit two's complement is

1111 1111 1111 1100 (= 0xffffc)

Substituting all three placeholder runs into the encoding:

000100 01000 10000 1111111111111100

yields the 32-bit machine code 00010001000100001111111111111100, or 0x1110ffffc in hexadecimal.

7.4 Jump: j 8

Looking up the syntax and encoding of j in `instructions.txt`:

- Syntax: j imm
- Encoding: 0000 10ii iiiiiiii iiiiiiii iiiiiiii iiiiiiii

The single operand 8 is the target byte address. Per Section 3, the value placed in the 26-bit i field is $8/4 = 2$, written as a 26-bit binary number:

000000000000000000000000000010

Substituting into the encoding:

000010 000000000000000000000000000010

yields 00001000000000000000000000000010, or 0x08000002. Note again that this is not 0x08000008 — the bottom two bits of the byte address are dropped.

8 Hints and Suggested Approach

A reasonable solution may be organised around the following ideas:

- **Parse the instruction syntax to discover the order of operands.** For example, the syntax `add $d, $s, $t` tells you that the first operand corresponds to the d field, the second to s, and the third to t. The first “character of interest” (s, t, d, or i) of each operand identifies the placeholder in the encoding.
- **Determine each field’s width from the encoding.** For each placeholder character, count its consecutive occurrences (ignoring spaces) to determine the field’s bit-width.
- **Substitute placeholders with binary values.** Replace the bits in the encoding template with the binary representation of the corresponding operand value, padded to the correct width.
- **Treat punctuation as separators.** Parentheses and commas (e.g. in `lw $t, offset($s)`) are merely separators; do not attempt to attach special meaning to them.
- **Convert the final binary string to hexadecimal** only after all substitutions are complete.

9 Constraints and Restrictions

Solutions that violate these constraints will receive **zero credit**, even if their output happens to be correct.

1. **No external utilities for binary conversion.** You must implement integer-to-binary and binary-to-hex conversion yourself. **Specifically, you must complete the functions `int_to_binary()` and `binary_to_hex()` in `binary.c`.**

You may not use any standard or third-party utility that produces a binary or hexadecimal representation for you (e.g. non-standard `printf/scanf` format specifiers for binary, the `%x` specifier for the final hex output, `itoa`-style routines, or any analogous library function). Standard I/O for reading files and writing the final output is permitted; basic string-manipulation utilities such as `strncpy`, `strchr`, `strtok`, and `atoi` are also permitted.

2. **Operand positions must be determined dynamically.** Do not branch on the instruction mnemonic. The following style is **not acceptable**:

```
if (strcmp(opcode, "lw") == 0) { ... }
```

Your code must derive operand placement entirely from the syntax and encoding read at run time from `instructions.txt`.

3. **Do not hard-code the instruction set.** The grader **will** test your assembler with instructions *not* listed in the sample `instructions.txt`. Your code must correctly handle any instruction whose syntax and encoding follow the same conventions. For example, your assembler should produce correct machine code for an entirely fictional instruction such as

```
abc $s, imm    with encoding    1110 10ss sssi iiii iiii iiii iiii iiii
```

even though your code has never seen the mnemonic `abc` before.

4. **Implementation language.** Your program must be implemented in **C**, conforming to the C11 standard. Your code must compile with `gcc -std=c11 -Wall -Wextra -Wpedantic` without errors.
5. **Command-line arguments.** The input files must be parsed as command-line arguments, in this order:

```
<executable> registers.txt instructions.txt program.txt
```

You can use the provided `Makefile` within the starter code to compile your code:

```
$ make
```

After compilation, you can run the program as follows:

```
$ ./assembler registers.txt instructions.txt program.txt
```

10 Submission and Grading

- **You MUST use this starter code**. All classes should be placed directly in your **zip** archive.
- Do not miss the deadline: **Saturday, 16.05.2026(23:59:59)**.
- You must submit your work via <https://submit.cs.hacettepe.edu.tr/> with the file hierarchy given below:

```
- b<studentID>.zip
  * assembler.h <FILE>
  * assembler.c <FILE>
  * binary.h <FILE>
```

```
* binary.c <FILE>
* main.c <FILE>
```

- You must test your code via **Tur⁶Bo Grader** <https://test-grader.cs.hacettepe.edu.tr/> (**does not count as submission!**).
- The grader compares your program's standard output against the expected output *character by character*. Be careful with trailing whitespace, blank lines, and the case of hex digits.
- You will be tested on inputs that exercise registers, instructions, and field layouts *not* present in the samples. Design your code accordingly.
- Late submissions, plagiarised code, or use of solutions found online will be handled in accordance with the BBM234 academic-honesty policy.

Challenge With Tiny Awards

The first **three** students who

- successfully complete the project (get 100 points on the **Tur⁶Bo Grader**),
- get the same score with the grading tests, and
- do not get flagged for plagiarism

will be rewarded with tiny awards.



Good luck — and have fun assembling.